

CS FUNDAMENTALS



Time & Space Complexity



A step-by-step guide for the student who finally wants to actually understand this stuff.

Big-O Notation

Time Complexity

Space Complexity

Analysis

WHY DOES THIS MATTER?

❓ Two programs both solve the same problem — which one is better?



100 items

Both programs finish in 0.001 seconds



10,000 items

Program A: 0.1s Program B: 10s



1,000,000 items

Program A: 1s Program B: NEVER
finishes!



Complexity tells you HOW your algorithm scales as input grows.

WHAT IS BIG-O NOTATION?



Simple Definition

Big-O describes the WORST CASE number of steps an algorithm takes as the input size (n) grows towards infinity.

$$f(n) = O(g(n))$$

Read as: "Order of g(n)"

The Pizza Analogy 🍕

n = number of slices to distribute

$O(1)$: Always cut the first slice — doesn't matter if pizza has 4 or 100 slices

$O(n)$: Hand one slice to each person — more people, more trips

$O(n^2)$: Every person hands a slice to every other person — chaos!

⚡ Drop constants & lower-order terms: $O(3n^2 + 2n + 1) = O(n^2)$

THE COMPLEXITY LADDER (Best → Worst)

$O(1)$	Constant	 Array index lookup	🏆 BEST
$O(\log n)$	Logarithmic	 Binary Search	✓ Excellent
$O(n)$	Linear	 Simple loop	👍 Good
$O(n \log n)$	Linearithmic	 Merge Sort	⚠️ Okay
$O(n^2)$	Quadratic	 Bubble Sort, Nested loops	🐢 Slow
$O(2^n)$	Exponential	 Recursive fibonacci	🔥 BAD

💡 For the same problem, always aim for the complexity closest to the top of this list.

$O(1)$ — CONSTANT TIME

Definition

The algorithm always takes the **SAME** number of steps, no matter how big the input is.

```
int arr[] = {10, 20, 30, 40, 50};  
// Access first element:  
int x = arr[0]; // Always 1 step
```

No matter if array has 5 or 5,000,000 items
→ `arr[0]` always takes exactly 1 step!

Real World Examples



Opening a book to a page number you already know



ATM gives you cash — doesn't matter how full the vault is

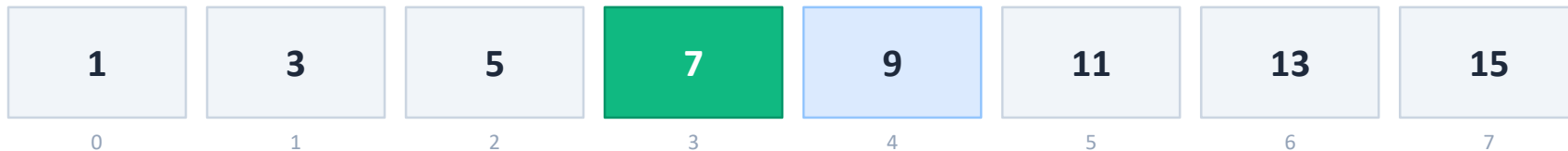


Looking up a dictionary word by page number (you already have it)

KEY INSIGHT: The number of steps is **FIXED**.
It does **NOT** change as n grows.

$O(\log n)$ — LOGARITHMIC TIME

Binary Search in Action: Find 7 in a sorted array



Step 1

Check middle element: 7 or 9 depending on rounding

Array size $\rightarrow$ 8

Step 2

$7 < 9$ (mid), so search LEFT half

Array size $\rightarrow$ 4

Step 3

Check middle of left half $\rightarrow$ Found 7! ✓

Array size $\rightarrow$ 2

The Math

$n = 8$ elements

Steps needed = $\log_2(8) = 3$

$n = 1,000,000$ elements

Steps needed = $\log_2(1M) \approx 20$

💡 Key Insight

Every time n DOUBLES, you only add 1 extra step.

That's why binary search is so powerful — even 1 BILLION elements only needs about 30 steps!

$O(n)$ — LINEAR TIME

```
// Find a value in unsorted array
int findValue(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target)
            return i; // Found!
    }
    return -1; // Not found
}
```

← *This loop runs n times!*

Trace (worst case — target not found):

$i=0$: $\text{arr}[0]=10 \neq 99$

$i=1$: $\text{arr}[1]=25 \neq 99$

$i=2$: $\text{arr}[2]=7 \neq 99$

$i=3$: $\text{arr}[3]=18 \neq 99$ (n steps total)

How Steps Grow with n

$n = 10$

10

$n = 100$

100

$n = 1,000$

1,000

$n = 10,000$

10,000

$n = 100,000$

100,000

Steps grow PROPORTIONALLY to input size.
Double the input → Double the steps.

$O(n^2)$ — QUADRATIC TIME (The Danger Zone)

```
// Bubble Sort — classic  $O(n^2)$ 
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                // swap
            }
        }
    }
}
```

← Outer loop: runs n times

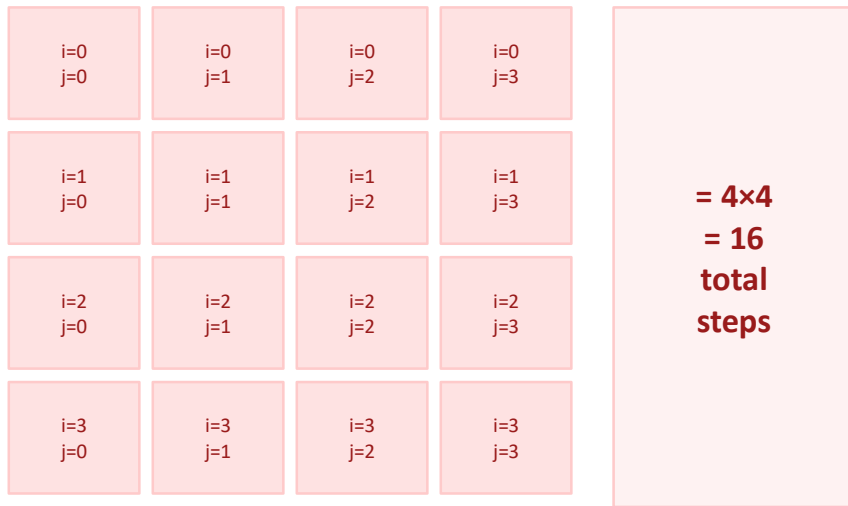
← Inner loop: also runs n times

Total = $n \times n = n^2$ comparisons

$n=1,000$: 1,000,000 steps 🤔
 $n=10,000$: 100,000,000 steps 💀

Visualizing n^2 ($n = 4$)

For each of 4 outer steps, we do 4 inner steps:



 **RULE:** Every nested loop usually adds another 'n' to your complexity. Watch for nested loops!

HOW TO CALCULATE TIME COMPLEXITY (Step-by-Step)

1 Single statement = $O(1)$

```
int x = 5 + 3;
```

Any single line of code that doesn't depend on n is constant time.

2 One loop = $O(n)$

```
for (int i = 0; i < n; i++) { ... }
```

A single loop running n times is linear.

3 Nested loops = multiply

```
for(i..n) { for(j..n) { ... } }
```

Two nested loops = $O(n \times n) = O(n^2)$. Three = $O(n^3)$.

4 Sequential blocks = add, then simplify

```
 $O(n) + O(n^2) = O(n^2)$ 
```

Keep only the dominant (largest) term.

GOLDEN RULE: Only the FASTEST growing term matters. Constants and lower-order terms are dropped.

SPACE COMPLEXITY — Memory Usage

Space Complexity = the amount of EXTRA MEMORY an algorithm uses relative to the input size.

$O(1)$ Space

Constant — uses same memory regardless

```
int swap(int &a, int &b) {  
    int temp = a; // 1 var  
    a = b;  
    b = temp;  
}
```

Only 1 extra variable (temp). Input size doesn't matter.

$O(n)$ Space

Linear — memory grows with input

```
int* copy(int arr[], int n) {  
    int* newArr = new int[n];  
    // Copy all elements  
    return newArr;  
}
```

Creates a new array of size n . More input = more memory.

$O(n)$ Space (Recursion)

Call stack counts as space!

```
int factorial(int n) {  
    if (n==1) return 1;  
    return n * factorial(n-1);  
}
```

Each recursive call uses stack space. n calls = $O(n)$ space.

⚡ Always consider BOTH time AND space complexity. Sometimes trading memory for speed is a good deal!

PRACTICE: Analyze This Code

```
void analyze(int arr[], int n) {  
    // Part A  
    int x = arr[0];           // ①  
  
    // Part B  
    for (int i = 0; i < n; i++) { // ②  
        cout << arr[i];  
    }  
  
    // Part C  
    for (int i = 0; i < n; i++) { // ③  
        for (int j = 0; j < n; j++) {  
            cout << arr[i]+arr[j];  
        }  
    }  
}
```

Step-by-Step Analysis

① Part A

```
int x = arr[0];
```

Single statement — constant time

$O(1)$

② Part B

```
for(i < n) { ... }
```

One loop over all n elements

$O(n)$

③ Part C

```
for(i<n) { for(j<n) }
```

Nested loop — $n \times n$ iterations

$O(n^2)$

$$O(1) + O(n) + O(n^2) = \mathbf{O(n^2)}$$

THE COMPLEXITY CHEAT SHEET

TIME COMPLEXITY

SPACE COMPLEXITY

Notation	Name	Example	Space	What Uses It
$O(1)$	Constant	Array access, swap	$O(1)$	A few variables
$O(\log n)$	Logarithmic	Binary Search	$O(n)$	New array / recursion
$O(n)$	Linear	Linear Search, single loop	$O(n^2)$	2D matrix
$O(n \log n)$	Linearithmic	Merge Sort, Heap Sort		
$O(n^2)$	Quadratic	Bubble Sort, nested loops		
$O(2^n)$	Exponential	Recursive subsets		

THE 4 GOLDEN RULES

1. Drop constants: $O(3n) = O(n)$

2. Drop lower terms: $O(n^2 + n) = O(n^2)$

3. Nested loop = multiply: $n \times n = n^2$

4. Recursion stack = $O(\text{depth})$



You now understand

Time & Space Complexity



Big-O measures worst-case
scaling behavior



Always aim for the lowest
complexity possible



Space & time are a tradeoff
— know both

Next: Practice on LeetCode with Easy → Medium problems